Diagrama de clases de diseño — Club Deportivo

modelo.actividades (instalaciones y eventos)
«abstract» Estadistica
- fechaRegistro: LocalDate
+ getValores(): Map<String,Integer>
+ calcularPuntosRanking(): int
EstadisticaFutbol
- goles: int
- asistencias: int
+ calcularPuntosRanking(): int
EstadisticaBaloncesto
- puntos: int
- rebotes: int
+ calcularPuntosRanking(): int
EstadisticaVoleibol
- aces: int
- bloqueos: int
+ calcularPuntosRanking(): int
EstadisticaIndividual
- puntosAnotados: int
- gano: boolean
+ calcularPuntosRanking(): int
«abstract» Evento
- id: String
- fecha: LocalDate
- horaInicio: LocalTime
- duracionMinutos: int
+ getHoraFin(): LocalTime
+ seTraslapaCon(f: LocalDate, ini: LocalTime, fin: LocalTime): boolean
+ bloqueaInstalacion(): boolean
ocupa
Instalacion
- codigo: String
- nombre: String
- capacidadMaxima: int
- horaApertura: LocalTime
- horaCierre: LocalTime
- enMantenimiento: boolean
+ estaDisponible(fecha: LocalDate, inicio: LocalTime, fin: LocalTime): boolean
+ admiteCantidad(n: int): boolean
+ agregarOcupacion(e: Evento): void
+ liberar(e: Evento): void
estadisticas
«abstract» Partido
- estado: EstadoPartido
- marcadorPropio: int
- marcadorRival: int
+ agregarParticipante(s: Socio): void
+ participo(s: Socio): boolean
+ registrarResultado(propio: int, rival: int): void
+ getResultado(): Resultado
+ agregarEstadistica(e: Estadistica): void
+ getDeporte(): Deporte
+ bloqueaInstalacion(): boolean
Reserva
- cantidadJugadores: int
- modalidad: ModalidadReserva
- estado: EstadoReserva
+ cancelar(): void
+ bloqueaInstalacion(): boolean
SesionEntrenamiento
- objetivo: String
- cancelada: boolean
+ estaConvocado(s: Socio): boolean
+ registrarAsistencia(s: Socio, e: EstadoAsistencia, nota: Integer, comentario: String): RegistroAsistencia
+ bloqueaInstalacion(): boolean
partidos
representa
registros
RegistroAsistencia
- estado: EstadoAsistencia
- notaDesempeno: Integer [0..1] {1..5}
- comentario: String [0..1]
+ tieneNota(): boolean
jugador
participantes
PartidoPractica
- nivelesVerificados: boolean
+ incluyeEntrenador(): boolean
+ getDeporte(): Deporte
PartidoOficial
- rival: String
- esLocal: boolean
- fase: String
+ getPuntosObtenidos(): int
+ getDeporte(): Deporte
hecha por
convocados
nominaOficial
programada por
jugadores
modelo.competencias (competencias y desempeño)
Competencia
- codigo: String
- nombre: String
- organizador: String
- categoria: Categoria
- formato: FormatoCompetencia
- estado: EstadoCompetencia
- fechaInicio: LocalDate
- fechaFin: LocalDate
- cierreInscripciones: LocalDate
- puntosVictoria: int
- puntosEmpate: int
- puntosDerrota: int
+ inscripcionesAbiertas(fecha: LocalDate): boolean
+ puntosPor(r: Resultado): int
+ agregarInscripcion(i: InscripcionCompetencia): void
+ agregarPartido(p: PartidoOficial): void
+ getTablaPosiciones(): List<InscripcionCompetencia>
Ranking
- ultimaActualizacion: LocalDate
+ actualizar(e: Estadistica): void
+ getEntrada(s: Socio): EntradaRanking
+ getPosiciones(): List<EntradaRanking>
inscripciones
entradas
InscripcionCompetencia
- fechaInscripcion: LocalDate
- puntosAcumulados: int
- partidosJugados: int
- eliminada: boolean
+ estaEnNomina(s: Socio): boolean
+ sumarPuntos(p: int): void
EntradaRanking
- partidosJugados: int
- puntosRanking: int
- acumulados: Map<String,Integer>
+ acumular(e: Estadistica): void
es para
disciplina
de
para
equipo
modelo.personal
SolicitudCambioTurno
- fechaSolicitud: LocalDate
- motivo: String
- estado: EstadoSolicitud
- fechaRespuesta: LocalDate
- observacion: String
+ aprobar(a: Administrador, fecha: LocalDate): void
+ rechazar(a: Administrador, obs: String, fecha: LocalDate): void
turnoPropuesto
Turno
- id: String
- fecha: LocalDate
- horaInicio: LocalTime
- horaFin: LocalTime
+ cubre(fecha: LocalDate, hora: LocalTime): boolean
+ seTraslapaCon(otro: Turno): boolean
solicitante
asignado a
revisada por
modelo.personas
«abstract» Persona
- documento: String
- nombre: String
- fechaNacimiento: LocalDate
- telefono: String
- correo: String
- login: String
- contrasena: String
+ getEdad(fecha: LocalDate): int
+ verificarContrasena(clave: String): boolean
+ getTipo(): String
«abstract» Empleado
- codigoEmpleado: String
- fechaContratacion: LocalDate
+ getTipo(): String
Administrador
- cargo: String
Fisioterapeuta
- tarjetaProfesional: String
Entrenador
- licencia: String
+ puedeEntrenar(d: Deporte): boolean
Socio
- numeroSocio: String
- fechaAfiliacion: LocalDate
- activo: boolean
+ getNivel(d: Deporte): int
+ asignarNivel(d: Deporte, nivel: int): void
+ estaSuspendido(d: Deporte, fecha: LocalDate): boolean
+ agregarSancion(s: Sancion): void
+ getCuenta(): CuentaFidelizacion
+ getTipo(): String
niveles
NivelDeportivo
- nivel: int {1..10}
- fechaActualizacion: LocalDate
+ diferenciaCon(otro: NivelDeportivo): int
sanciones
Sancion
- motivo: String
- tipo: TipoSancion
- fechaInicio: LocalDate
- fechaFin: LocalDate
+ estaVigente(fecha: LocalDate): boolean
+ aplicaA(d: Deporte): boolean
comprador
dirigido por
especialidades
a cargo de
alumno
jugador
realizado por
cuenta
modelo.deportes
Deporte
- nombre: String
- esDeConjunto: boolean
- jugadoresEnCampo: int
- minJugadoresReserva: int
- maxJugadoresReserva: int
- minJugadoresNomina: int
- maxJugadoresNomina: int
- duracionPartidoMin: int
- permiteEmpate: boolean
- diferenciaNivelPermitida: int
- posiciones: List<String>
- tipoEstadistica: TipoEstadistica
+ cantidadReservaValida(n: int): boolean
+ nominaValida(n: int): boolean
+ esPosicionValida(p: String): boolean
+ nivelesCompatibles(a: int, b: int): boolean
+ crearEstadistica(j: Socio, p: Partido, v: Map<String,Integer>): Estadistica
en
aplica en
practica
de
Equipo
- nombre: String
- categoria: Categoria
- cupoMaximo: int
+ tieneCupo(): boolean
+ esIntegrante(s: Socio): boolean
+ inscribir(s: Socio, posicion: String, numero: int, fecha: LocalDate): InscripcionEquipo
+ getJugadoresActivos(): List<Socio>
Academia
- nombre: String
- nivelMinimo: int
- nivelMaximo: int
- cupoMaximo: int
- horario: String
- valorMensualidad: double
+ tieneCupo(): boolean
+ admiteNivel(n: int): boolean
+ inscribir(s: Socio, fecha: LocalDate): InscripcionAcademia
inscripciones
InscripcionEquipo
- fechaInscripcion: LocalDate
- posicion: String
- numeroCamiseta: int
- activa: boolean
+ retirar(): void
InscripcionAcademia
- fechaInscripcion: LocalDate
- activa: boolean
+ retirar(): void
academia
pagada por
modelo.inventario
Inventario
+ estaRegistrado(p: Producto): boolean
+ getCantidad(p: Producto, u: Ubicacion): int
+ consultar(u: Ubicacion): List<Existencia>
+ obtenerExistencia(p: Producto, u: Ubicacion): Existencia
+ reabastecer(p: Producto, u: Ubicacion, cant: int, a: Administrador): Reabastecimiento
+ trasladar(p: Producto, origen: Ubicacion, destino: Ubicacion, cant: int, a: Administrador): Traslado
+ descontar(p: Producto, u: Ubicacion, cant: int): void
existencias
movimientos
ubicaciones
Existencia
- cantidad: int
- stockMinimo: int
+ aumentar(n: int): void
+ disminuir(n: int): void
+ estaBajoMinimo(): boolean
«abstract» MovimientoInventario
- fechaHora: LocalDateTime
- cantidad: int
+ aplicar(inv: Inventario): void
producto
Reabastecimiento
+ aplicar(inv: Inventario): void
Traslado
+ aplicar(inv: Inventario): void
en
Ubicacion
- codigo: String
- nombre: String
- tipo: TipoUbicacion
destino
origen
destino
ubicado en
modelo.comercio (tienda, cafetería y fidelización)
Venta
- numero: String
- fechaHora: LocalDateTime
- subtotal: double
- descuento: double
- impuesto: double
- propina: double
- puntosRedimidos: int
- valorRedimido: double
- totalPagado: double
- puntosGanados: int
- alergenosInformados: boolean
- cerrada: boolean
+ agregarItem(p: Producto, cantidad: int): void
+ requiereInfoAlergenos(): boolean
+ marcarAlergenosInformados(): void
+ calcularTotales(pv: PuntoVenta): void
+ aplicarPropina(valor: double): void
+ redimirPuntos(puntos: int, valor: double): void
+ getBaseFidelizacion(): double
+ cerrar(): void
ventas
«abstract» PuntoVenta
- nombre: String
+ admiteProducto(p: Producto): boolean
+ calcularDescuento(c: Persona, subtotal: double): double
+ calcularImpuesto(baseGravable: double): double
+ admitePropina(): boolean
+ registrarVenta(v: Venta): void
CuentaFidelizacion
- puntos: int
- PESOS_POR_PUNTO: int = 1000
- VALOR_PUNTO: double = 10
+ acumular(totalPagado: double): int
+ puedeRedimir(p: int): boolean
+ redimir(p: int): double
Mensualidad
- periodo: YearMonth
- concepto: ConceptoMensualidad
- valor: double
- fechaPago: LocalDate
+ getRubro(): Rubro
items
producto
«abstract» Producto
- codigo: String
- nombre: String
- precio: double
+ getRubro(): Rubro
ItemVenta
- cantidad: int
- precioUnitario: double
+ getSubtotal(): double
Tienda
- TASA_IVA: double = 0.19
- DESCUENTO_EMPLEADO: double = 0.15
- DESCUENTO_SOCIO: double = 0.08
+ admiteProducto(p: Producto): boolean
+ calcularDescuento(c: Persona, subtotal: double): double
+ calcularImpuesto(baseGravable: double): double
+ admitePropina(): boolean
Cafeteria
- TASA_IMPOCONSUMO: double = 0.08
- PROPINA_SUGERIDA: double = 0.10
+ admiteProducto(p: Producto): boolean
+ calcularDescuento(c: Persona, subtotal: double): double
+ calcularImpuesto(baseGravable: double): double
+ admitePropina(): boolean
+ sugerirPropina(subtotal: double): double
ArticuloDeportivo
- marca: String
- talla: String
+ getRubro(): Rubro
Snack
- calorias: int
+ getRubro(): Rubro
+ getAlergenos(): List<Alergeno>
+ contieneAlergeno(a: Alergeno): boolean
ingredientes
Ingrediente
- nombre: String
+ getAlergenos(): List<Alergeno>
alergenos
Alergeno
- nombre: String
- descripcion: String
logica
GestorUsuarios
- personas: Map<String, Persona>
+ autenticar(login: String, clave: String): Persona
+ registrarSocio(s: Socio): void
+ registrarEmpleado(e: Empleado): void
+ buscarSocio(documento: String): Socio
+ getEntrenadores(): List<Entrenador>
+ getFisioterapeutas(): List<Fisioterapeuta>
+ sancionar(s: Socio, motivo: String, t: TipoSancion, ini: LocalDate, fin: LocalDate, d: Deporte): Sancion
GestorInstalaciones
- instalaciones: List<Instalacion>
+ buscarDisponible(d: Deporte, f: LocalDate, ini: LocalTime, dur: int, n: int): Instalacion
+ reservar(s: Socio, d: Deporte, f: LocalDate, ini: LocalTime, dur: int, n: int, m: ModalidadReserva): Reserva
+ cancelarReserva(r: Reserva): void
+ consultarDisponibilidad(d: Deporte, f: LocalDate): Map<Instalacion, List<LocalTime>>
instalaciones
GestorDeportivo
- deportes: List<Deporte>
- equipos: List<Equipo>
- academias: List<Academia>
- sesiones: List<SesionEntrenamiento>
- partidosPractica: List<PartidoPractica>
- rankings: Map<Deporte, Ranking>
+ inscribirEnEquipo(s: Socio, e: Equipo, posicion: String, numero: int): InscripcionEquipo
+ inscribirEnAcademia(s: Socio, a: Academia): InscripcionAcademia
+ programarEntrenamiento(en: Entrenador, f: LocalDate, ini: LocalTime, dur: int, i: Instalacion, conv: List<Socio>): SesionEntrenamiento
+ registrarAsistencia(en: Entrenador, se: SesionEntrenamiento, s: Socio, e: EstadoAsistencia, nota: Integer, com: String): void
+ programarPartidoPractica(sol: Socio, rival: Persona, d: Deporte, f: LocalDate, ini: LocalTime): PartidoPractica
+ registrarEstadistica(en: Entrenador, p: Partido, j: Socio, v: Map<String,Integer>): Estadistica
+ consultarRanking(d: Deporte): List<EntradaRanking>
- buscarEquipoDe(s: Socio, d: Deporte): Equipo
deportivo
GestorPersonal
- turnos: List<Turno>
- solicitudes: List<SolicitudCambioTurno>
- MIN_ENTRENADORES: int = 2
- MIN_FISIOTERAPEUTAS: int = 1
- horaApertura: LocalTime
- horaCierre: LocalTime
+ asignarTurno(t: Turno): void
+ solicitarCambio(e: Empleado, actual: Turno, propuesto: Turno, motivo: String): SolicitudCambioTurno
+ aprobarCambio(s: SolicitudCambioTurno, a: Administrador): void
+ rechazarCambio(s: SolicitudCambioTurno, a: Administrador, obs: String): void
+ coberturaMinimaCumplida(turnos: List<Turno>, f: LocalDate): boolean
- contarDisponibles(turnos: List<Turno>, f: LocalDate, h: LocalTime, tipo: Class): int
ClubDeportivo
- nombre: String
- nit: String
- direccion: String
- horaApertura: LocalTime
- horaCierre: LocalTime
+ iniciarSesion(login: String, clave: String): Persona
+ getGestorUsuarios(): GestorUsuarios
+ getGestorInstalaciones(): GestorInstalaciones
+ getGestorDeportivo(): GestorDeportivo
+ getGestorCompetencias(): GestorCompetencias
+ getGestorComercio(): GestorComercio
+ getGestorPersonal(): GestorPersonal
+ getGestorReportes(): GestorReportes
GestorCompetencias
- competencias: List<Competencia>
+ registrarCompetencia(c: Competencia): void
+ inscribirEquipo(c: Competencia, e: Equipo, nomina: List<Socio>): InscripcionCompetencia
+ inscribirIndividual(c: Competencia, s: Socio): InscripcionCompetencia
+ programarPartido(c: Competencia, i: InscripcionCompetencia, rival: String, f: LocalDate, ini: LocalTime, local: boolean): PartidoOficial
+ registrarResultado(p: PartidoOficial, propio: int, rival: int): void
+ consultarTabla(c: Competencia): List<InscripcionCompetencia>
ReporteVentas
- fechaInicio: LocalDate
- fechaFin: LocalDate
- totalesPorRubro: Map<Rubro, Double>
- totalesPorMes: Map<YearMonth, Map<Rubro, Double>>
- transacciones: int
+ agregar(r: Rubro, mes: YearMonth, valor: double): void
+ getTotal(): double
+ estaVacio(): boolean
«crea»
GestorReportes
+ generarReporteVentas(ini: LocalDate, fin: LocalDate): ReporteVentas
comercio
GestorComercio
- tienda: Tienda
- cafeteria: Cafeteria
- inventario: Inventario
- productos: Map<String, Producto>
- alergenos: List<Alergeno>
- mensualidades: List<Mensualidad>
- consecutivoVentas: int
+ iniciarVenta(pv: PuntoVenta, comprador: Persona): Venta
+ agregarProducto(v: Venta, p: Producto, cant: int): void
+ consultarIngredientes(s: Snack): List<Ingrediente>
+ finalizarVenta(v: Venta, propina: double, puntos: int): Venta
+ registrarMensualidad(s: Socio, periodo: YearMonth, c: ConceptoMensualidad, valor: double): Mensualidad
+ buscarSnacksConAlergeno(a: Alergeno): List<Snack>
+ getVentas(ini: LocalDate, fin: LocalDate): List<Venta>
+ getMensualidades(ini: LocalDate, fin: LocalDate): List<Mensualidad>
+ getInventario(): Inventario
club
#club
«carga/salva»
consola
ConsolaPrincipal
+ main(args: String[]): void
- iniciarSesion(): void
- abrirConsola(p: Persona): void
persistencia
«crea»
«abstract» ConsolaBase
# pedirCadena(msg: String): String
# pedirEntero(msg: String): int
# pedirFecha(msg: String): LocalDate
+ mostrarMenu(): void
ConsolaSocio
- socio: Socio
+ mostrarMenu(): void
ConsolaEntrenador
- entrenador: Entrenador
+ mostrarMenu(): void
ConsolaFisioterapeuta
- fisio: Fisioterapeuta
+ mostrarMenu(): void
ConsolaAdministrador
- admin: Administrador
+ mostrarMenu(): void
persistencia
«interface» IPersistenciaClub
+ cargar(ruta: String): ClubDeportivo
+ salvar(club: ClubDeportivo, ruta: String): void
PersistenciaJson
+ cargar(ruta: String): ClubDeportivo
+ salvar(club: ClubDeportivo, ruta: String): void
excepciones
«exception» ClubException
+ ClubException(msg: String)
«exception» ReservaException
«exception» InscripcionException
«exception» ParticipacionException
«exception» VentaException
«exception» InventarioException
«exception» TurnoException
Enumeraciones (tipos de atributo)
«enumeration» TipoSancion
AMONESTACION
SUSPENSION_DEPORTIVA
SUSPENSION_TOTAL
«enumeration» Categoria
SUB_13 (0, 13)
SUB_15 (14, 15)
SUB_17 (16, 17)
MAYORES (18, 120)
- edadMinima: int
- edadMaxima: int
+ paraEdad(edad: int): Categoria
«enumeration» TipoEstadistica
FUTBOL
BALONCESTO
VOLEIBOL
INDIVIDUAL
«enumeration» ModalidadReserva
INDIVIDUAL
DOBLES
GRUPAL
«enumeration» EstadoReserva
CONFIRMADA
CANCELADA
FINALIZADA
«enumeration» EstadoAsistencia
ASISTIO
FALTO
LLEGO_TARDE
«enumeration» EstadoPartido
PROGRAMADO
FINALIZADO
CANCELADO
«enumeration» Resultado
VICTORIA
EMPATE
DERROTA
«enumeration» FormatoCompetencia
LIGA
ELIMINACION_DIRECTA
GRUPOS_Y_ELIMINACION
«enumeration» EstadoCompetencia
INSCRIPCIONES_ABIERTAS
EN_CURSO
FINALIZADA
«enumeration» Rubro
ARTICULOS_DEPORTIVOS
CAFETERIA
MENSUALIDADES
«enumeration» ConceptoMensualidad
AFILIACION
ACADEMIA
«enumeration» TipoUbicacion
BODEGA
TIENDA
CAFETERIA
«enumeration» EstadoSolicitud
PENDIENTE
APROBADA
RECHAZADA